AdaGuard: Leakage-Aware Adaptive Encryption for Privacy-Preserving Federated Learning

Firas Aguir
Department of Computer Science, Oakland University, Rochester, MI, USA

Abstract

Federated Learning (FL) enables collaborative model training without centralizing raw data, yet recent gradient inversion attacks have demonstrated that shared model updates can leak sensitive training information. Existing defenses either apply uniform encryption to all parameters—incurring prohibitive computational and communication overhead—or rely on heuristic noise injection that degrades model utility. We present **AdaGuard**, a framework that continuously quantifies gradient leakage risk through a multi-component **LeakScore** metric and dynamically allocates selective homomorphic encryption only to the most vulnerable model parameters. LeakScore combines three complementary signals: (i) an entropy-based measure of gradient distribution concentration, (ii) a label leakage proxy capturing class-discriminative structure in gradients, and (iii) an empirical score derived from executing gradient inversion attacks in real time. AdaGuard further leverages per-weight Fisher Information to rank parameter sensitivity and applies encryption to the top-k most informative weights, adapting k each round based on the observed LeakScore. We compare our Fisher-based selection against MaskCrypt's vulnerability-weighted scheme and evaluate on CIFAR-10 with up to 1,000 non-IID clients using ResNet-18 on a multi-GPU HPC cluster. Our results demonstrate that AdaGuard achieves comparable privacy protection to full encryption while encrypting only 10–15% of parameters, reducing computational cost by up to 8x and communication overhead by up to 6x, with less than 1% degradation in test accuracy.

I. Introduction

Federated Learning (FL) [1] has emerged as a foundational paradigm for privacy-preserving machine learning, enabling multiple clients to collaboratively train a shared model by exchanging model updates—typically gradients or weight deltas—rather than raw data. This architecture has found adoption across domains where data sensitivity is paramount, including healthcare [2], finance [3], and mobile computing [4]. By keeping training data on-device, FL ostensibly provides a privacy guarantee: the server never observes the raw samples.

However, a growing body of work has shattered this assumption. Zhu et al. [5] demonstrated that shared gradients can be inverted to reconstruct training images with pixel-level fidelity through optimization-based attacks. Subsequent work has refined these attacks: Zhao et al. [6] showed that labels can be analytically extracted from the gradient of the last fully-connected layer (the iDLG theorem), while Geiping et al. [7] introduced cosine-similarity objectives that improve reconstruction quality for larger batch sizes. More recent attacks leverage neural architecture search (GI-NAS) [8] and diffusion-inspired noise annealing (GGCDM) [9] to further push the frontier of gradient inversion. These

developments establish that **sharing gradients is not inherently private**, and that meaningful defenses are necessary.

The most straightforward defense is to encrypt all shared parameters using Homomorphic Encryption (HE), which permits aggregation on encrypted values without decryption [10]. While HE provides strong cryptographic guarantees, it introduces severe overhead: ciphertext expansion ratios of 50–100x increase communication costs, and HE operations are 3–6 orders of magnitude slower than their plaintext counterparts [11]. For modern architectures with tens of millions of parameters, full HE encryption is impractical for real-time FL deployments.

Alternative defenses such as differential privacy (DP) [12] add calibrated noise to gradients before sharing. While DP provides formal privacy guarantees, the noise magnitude required for meaningful protection often substantially degrades model accuracy, particularly in non-IID settings where clients hold heterogeneous data distributions [13]. Secure aggregation protocols [14] prevent the server from observing individual updates but require complex multi-party communication and do not protect against a compromised aggregator.

A key insight motivates our approach: **not all parameters are equally vulnerable to gradient inversion**. Fisher Information theory tells us that the squared gradient magnitude $F_i = g_i^2$ quantifies how much information parameter i carries about the training data [15]. In practice, Fisher Information is highly concentrated—a small fraction of weights (often less than 5%) carries the majority of the information. This concentration, measured by the Gini coefficient of the Fisher distribution, suggests that **selective encryption** of only the most informative parameters can achieve comparable privacy protection at a fraction of the cost.

Building on this insight, we introduce **AdaGuard**, a leakage-aware adaptive encryption framework for federated learning. AdaGuard makes three key contributions:

- **LeakScore Framework.** A composite metric that quantifies gradient leakage risk in real time by combining entropy-based distribution analysis, label leakage proxies, and empirical gradient inversion attack scores. Unlike static privacy budgets, LeakScore adapts to the actual leakage characteristics of each round.
- **Adaptive Selective Encryption.** A three-tier encryption policy (none/partial/strong) that dynamically adjusts the fraction of encrypted parameters based on LeakScore. The policy uses Fisher Information to rank parameter sensitivity and encrypts only the top-k most vulnerable weights, where k is proportional to the detected leakage level.
- **Comprehensive Evaluation.** We compare Fisher-based parameter selection against MaskCrypt [16], a recently proposed vulnerability-weighted encryption scheme, and evaluate both against full encryption and no encryption baselines. Experiments on CIFAR-10 with non-IID partitioning across up to 1,000 clients demonstrate that AdaGuard encrypts 10–15% of parameters while maintaining privacy protection comparable to full encryption.

The remainder of this paper is organized as follows. Section II discusses related work. Section III presents the AdaGuard framework in detail, including the LeakScore formulation, adaptive encryption policy, and Fisher-based parameter selection. Section IV describes our experimental setup. Section V reports results. Section VI discusses implications and limitations. Section VII concludes.

III. AdaGuard

We now present the AdaGuard framework. We begin with an overview of the system architecture, then detail the LeakScore metric, the adaptive encryption policy, and the two parameter selection strategies (Fisher-based and MaskCrypt-based).

A. System Overview

AdaGuard operates within a standard FL setting: a central server coordinates T rounds of training across N clients, each holding a private dataset D_i. In each round t, the server selects a subset S_t of K clients, broadcasts the current global model w_t, and each selected client performs local SGD training to produce a weight update $delta_i = w_t - w_{i,t}$. Before transmitting $delta_i$ to the server, AdaGuard intercepts the update and performs three operations:

- **Measure:** Compute LeakScore($delta_i$) to quantify leakage risk.
- **Decide:** Apply the adaptive policy to determine encryption level and parameter selection.
- **Protect:** Selectively encrypt the identified parameters using homomorphic encryption.

The server then aggregates the (partially) encrypted updates via FedAvg [1] and updates the global model. This measure–decide–protect loop executes every round, allowing AdaGuard to adapt to changing leakage dynamics as training progresses.

B. LeakScore: Quantifying Gradient Leakage Risk

LeakScore is a composite metric that captures gradient leakage risk from three complementary perspectives. Each component produces a score in [0, 1], where higher values indicate greater risk. The final LeakScore is a weighted average:

$$\text{LeakScore} = (\alpha \cdot L_{\text{entropy}} + \beta \cdot L_{\text{label}} + \gamma \cdot L_{\text{empirical}}) \,/\, (\alpha + \beta + \gamma)$$

where α, β, γ are configurable weights (default: 1.0 each). We now detail each component.

B.1 Entropy LeakScore

The entropy component measures how concentrated the gradient distribution is. Prior work [5, 7] has shown that gradients with peaked, low-entropy distributions are more susceptible to inversion attacks because the optimization landscape for reconstruction is better conditioned. We preprocess the gradient vector through L2 normalization and standardization, then bin the result into a B-bin histogram (default B = 50) to obtain a discrete probability distribution p.

We compute three entropy variants and convert each to a leakage score:

$$\text{Shannon:} \quad H_{sh} = -\Sigma\, p_i \log(p_i), \quad \text{Score} = 1 - H_{sh} / \log(B)$$

$$\text{Renyi } (\alpha{=}2): \quad H_{re} = -\log(\Sigma\, p_i^2), \quad \text{Score} = 1 - H_{re} / \log(B)$$

$$\text{Min-Entropy:} \quad H_{min} = -\log(\max(p_i)), \quad \text{Score} = 1 - H_{min} / \log(B)$$

All scores are clamped to [0, 1]. A uniform gradient distribution yields maximum entropy (score 0, low risk), while a highly peaked distribution yields low entropy (score near 1, high risk). Following the iDLG theorem [6], we optionally restrict entropy computation to focus layers (e.g., the final fully-connected layer) where label information is most concentrated.

B.2 Label LeakScore

The label component quantifies how much class-label information is embedded in the gradient structure. We compute three complementary sub-metrics:

GLMIP (Gradient-Label Mutual Information Proxy). Inspired by Fisher's linear discriminant, GLMIP computes the ratio of between-class to total gradient variance. We partition gradients by their corresponding labels and compute:

$$S_B = \Sigma_c |C_c| \cdot ||\mu_c - \mu||^2 \quad \text{(between-class scatter)}$$

$$S_W = \Sigma_c \Sigma_{g \in C} ||g - \mu_c||^2 \quad \text{(within-class scatter)}$$

$$GLMIP = S_B / (S_B + S_W)$$

A GLMIP score near 1 indicates that gradients are highly separable by class, implying that an attacker can reliably extract label information.

Confidence Gap. We measure the gap between the top two softmax probabilities: Score = max(softmax) – second_max(softmax). High confidence correlates with gradient structure that reveals the true label.

Cosine Similarity. We compute the mean pairwise cosine similarity m between class-conditional gradient means, then map to a leakage score: Score = 1 – (m + 1)/2. Low inter-class similarity (dissimilar gradient signatures per class) indicates that class labels can be distinguished from the gradient.

B.3 Empirical LeakScore

The empirical component provides a direct measurement of leakage by executing gradient inversion attacks. Unlike entropy and label scores, which are proxies, this component measures actual reconstruction capability. We run three attacks with distinct optimization strategies:

- **GradInversion [5]:** Optimizes a dummy input to minimize $(1 - \cos\_sim(g_{recon}, g_{real})) + \lambda_{tv} \cdot TV(x)$, where TV(x) is total variation regularization.
- **GI-NAS [8]:** Uses L2 gradient matching with group-lasso regularization and multi-restart optimization (3 restarts, keeping the best result).
- **GGCDM [9]:** Combines cosine and L2 objectives with annealed noise injection: x += (1 – t/T) · 0.01 · noise, inspired by diffusion processes.

Each attack produces a reconstruction quality score:

$$score_{attack} = \max(0, \quad 1 - ||g_{real} - g_{recon}||_2 / ||g_{real}||_2)$$

The empirical LeakScore is the mean across all three attacks. A score near 1 indicates that at least one attack nearly perfectly matched the true gradient, implying successful data reconstruction. While computationally expensive, the empirical component serves as ground truth for calibrating the faster proxy metrics.

C. Adaptive Encryption Policy

AdaGuard uses a three-tier policy that maps LeakScore to an encryption level, parameterized by thresholds T_1 and T_2 (defaults: 0.3 and 0.7):

$$LeakScore < T_1: \quad level = none, \quad encrypt\_pct = 0\%$$

$$T_1 \leq \text{LeakScore} < T_2: \quad \text{level = partial}, \quad \text{encrypt\_pct scales linearly}$$

$$\text{LeakScore} \geq T_2: \quad \text{level = strong}, \quad \text{encrypt\_pct scales up to 100\%}$$

In the partial regime, the encryption percentage interpolates linearly between 0 and the base encryption rate (default 10%). In the strong regime, it continues scaling from the base rate toward full encryption. This smooth scaling avoids abrupt transitions and allows the system to apply minimal overhead when leakage risk is low while ramping up protection when risk is elevated.

The policy output determines k, the number of parameters to encrypt: k = encrypt_pct × n, where n is the total parameter count. The parameter selection strategy (Fisher or MaskCrypt) then identifies which specific k parameters to protect.

D. Fisher-Based Parameter Selection

Our primary parameter selection strategy uses the empirical Fisher Information to rank parameter sensitivity. The Fisher Information matrix approximation at parameter i is:

$$F_i = g_i^2$$

This diagonal approximation, while simple, captures the key insight: parameters with large squared gradients carry more information about the training batch and are thus more valuable to an attacker. We select the top-k parameters by F_i for encryption.

We additionally compute two aggregate statistics for monitoring:

$$\text{Fisher Concentration (Gini):} \quad G = (2 \cdot \Sigma\, i \cdot s_i) \,/\, (n \cdot \Sigma\, s_i) - (n+1)/n$$

where $s_1, \ldots, s_n$ are the Fisher values sorted in ascending order. A Gini coefficient near 1 indicates that information is concentrated in a small fraction of parameters, validating the selective encryption approach. We also compute a normalized Fisher score via sigmoid mapping: $f_{norm} = f_{round} / (f_{round} + 1)$, providing a scale-independent measure in [0, 1].

E. MaskCrypt-Based Parameter Selection

We additionally implement the MaskCrypt vulnerability scoring scheme proposed by Hu and Li [16] as a comparison baseline. MaskCrypt computes a per-weight vulnerability score that combines gradient magnitude with the change in exposed model weights across rounds:

$$v[i] = g[i] \cdot (w_{\text{exposed}}[i] - w_{\text{trained}}[i])$$

where $w_{exposed}$ represents the global model weights that were broadcast (and thus exposed to potential adversaries) in the previous round, and $w_{trained}$ represents the locally trained weights in the current round. The intuition is that parameters where both the gradient is large and the weight has changed significantly between rounds are most informative to an attacker who has observed the previous round's broadcast.

MaskCrypt selects the top-k parameters by |v[i]| for encryption. We evaluate its concentration via the top-fraction score: Score = Σ(top_k |v|) / Σ(all |v|), and its dispersion via the L2 score: $||v||_2$ / (√n · max|v|).

F. Gradient Magnitude Score

As a lightweight supplementary metric, we compute the total gradient L2 norm and map it to [0, 1] via smooth normalization:

$$\text{Score} = \min(1, \quad ||g||_2 / (||g||_2 + 1))$$

This metric captures the intuition that larger gradients carry more information and are more susceptible to inversion. While less nuanced than Fisher or entropy measures, it provides a fast sanity check and correlates with empirical attack success rates.

G. Computational Considerations

A critical advantage of selective encryption is reduced overhead. For a model with n parameters and encryption fraction p, the computational cost scales as $O(p \cdot n)$ rather than $O(n)$. With typical HE expansion factors of 50–100x, encrypting 10% of parameters reduces communication overhead by approximately 9x compared to full encryption. The LeakScore computation itself adds overhead: entropy and Fisher metrics are $O(n)$, while empirical attacks require multiple forward-backward passes. However, the empirical component can be computed on a configurable schedule (e.g., every 5 rounds) rather than every round, amortizing its cost.

For multi-GPU deployments, AdaGuard parallelizes client processing across available GPUs using thread-based concurrency, with each client assigned to a GPU in round-robin fashion. Metric computation is performed per-client on the assigned GPU to avoid cross-device memory transfers.

Writing Guidelines for Research Papers

The following guidelines are intended to help you maintain academic rigor and integrity as you complete the remaining sections of this paper. These principles apply to all sections.

1. General Principles

- **Claim only what you can prove.** Every factual claim must be backed by either a citation, your own experimental data, or a mathematical derivation. If you write "our method reduces cost by 8x," you must have the numbers to support it.
- **Distinguish contributions from prior work.** Clearly state what is yours versus what you are building upon. Use phrases like "we extend," "we adapt," or "building on [X], we propose."
- **Be precise with language.** Avoid vague quantifiers ("significantly better," "much faster"). Use exact numbers: "reduces overhead from 50x to 5.2x" is stronger than "greatly reduces overhead."
- **Acknowledge limitations honestly.** A paper that claims no limitations appears naive. State what your system does not handle (e.g., "our threat model assumes an honest-but-curious server").

2. Writing the Related Work Section

- **Organize thematically, not chronologically.** Group papers by topic: (a) gradient inversion attacks, (b) HE-based defenses, (c) DP-based defenses, (d) selective/adaptive encryption. End each group by explaining what gap remains.
- **Be fair to competitors.** Describe prior methods accurately before explaining why your approach differs. Avoid strawman comparisons.
- **Connect each reference to your work.** Every cited paper should serve a purpose: establishing the problem, motivating a design choice, or providing a comparison point.

3. Writing the Results Section

- **Present results before interpreting them.** State the numbers, then explain what they mean. "Fisher encryption at 10% achieves 0.92 LeakScore reduction (Table 2); this suggests that..."
- **Use tables for exact values, figures for trends.** A convergence curve belongs in a figure; a comparison of final accuracy across 4 strategies belongs in a table.
- **Report variance.** Run experiments multiple times (3–5 seeds) and report mean ± std. Single-run results are not publishable at top venues.
- **Structure around research questions.** Frame results as answers to specific questions: Q1: Does selective encryption stop attacks? Q2: Does it maintain accuracy? Q3: What is the cost reduction? Q4: How does Fisher compare to MaskCrypt?

4. Writing the Discussion Section

• **Interpret, do not repeat.** The discussion should explain *why* results look the way they do, not restate the numbers.

• **Connect to the broader field.** How do your findings advance the state of the art? What do they imply for practitioners deploying FL systems?

• **Address threats to validity.** Internal validity: could bugs or implementation choices affect results? External validity: does CIFAR-10 generalize to medical imaging? Construct validity: does LeakScore actually measure what we claim?

5. Writing the Threats to Validity Section

• **Construct validity:** Does LeakScore truly capture leakage risk? Validate by showing correlation between LeakScore and actual reconstruction quality (PSNR, SSIM, MSE). If they correlate, the metric is meaningful.

• **Internal validity:** Are results caused by AdaGuard or by confounds? Control for: random seed, data partitioning, model initialization. Use the same initial model across all strategies.

• **External validity:** CIFAR-10 is small and well-studied. Acknowledge that results may differ on larger datasets (ImageNet), different modalities (text, tabular), or different model architectures. Non-IID with max-2-classes is one specific heterogeneity pattern.

6. Citation Integrity

• **Cite primary sources.** Cite the original paper, not a survey or blog post that describes it.

• **Do not cite papers you have not read.** If you only know a result secondhand, either read the original or cite it as "as reported in [X]."

• **Use a consistent citation style.** IEEE format for this template: [1], [2], etc., with full bibliographic details in the References section.

7. Common Pitfalls to Avoid

• **Overclaiming.** "Our method is the first to..." requires exhaustive literature review. Safer: "To the best of our knowledge, no prior work has..."

• **Cherry-picking results.** Report all experiments, including ones where AdaGuard underperforms. Reviewers will run your code.

• **Ignoring baselines.** Always compare against: (a) no encryption, (b) full encryption, (c) differential privacy (if feasible), (d) MaskCrypt or other selective methods.

• **Vague experimental setup.** Specify every hyperparameter: learning rate, batch size, number of local steps, encryption percentage, number of clients, rounds, seeds. Reproducibility is non-negotiable.